

```
// =====
// CHEF PAPI - WhatsApp Ordering Backend v6
// Stack: Node.js + Express | Supabase | Meta WhatsApp Cloud API
// Features: AI FAQ, Dominican slang, NCF, human handoff,
//           session timeout, delivery zone check
// =====

const express = require("express");
const app = express();
app.use(express.json());

const {
  WHATSAPP_TOKEN,
  WHATSAPP_VERIFY_TOKEN,
  WHATSAPP_PHONE_ID,
  SUPABASE_URL,
  SUPABASE_KEY,
  ANTHROPIC_API_KEY,
  PORT = 3000,
} = process.env;

const { createClient } = require("@supabase/supabase-js");
const supabase = createClient(SUPABASE_URL, SUPABASE_KEY);

// — FLAVORS —————
const FLAVORS = {
  none: { title: "Sin Sabor", emoji: "🍷" },
  pomodoro: { title: "Pomodoro", emoji: "🍅" },
  pesto: { title: "Pesto", emoji: "🌿" },
  aglio: { title: "Aglio", emoji: "🧄" },
  teriyaki: { title: "Teriyaki", emoji: "🍡" },
  bbq: { title: "Barbecue", emoji: "🔥" },
};

// — PRICES —————
const PRICES = { 3: 900, 5: 1500, 10: 3000 };

// — SESSION TIMEOUT (minutes) —————
const SESSION_TIMEOUT_MINUTES = 10;

// — CHEF PAPI KNOWLEDGE BASE —————
const CHEF_PAPI_KNOWLEDGE = `
Eres el asistente virtual de Chef Papi, una marca de pollo cocinado al grill,
listo para comer, que se entrega frío en Santo Domingo, República Dominicana.

```

Tu personalidad: Eres simpático, dominicano, usas slang dominicano natural (como "diay", "qué lo qué", "ta bien", "brutal", "qué nivel").
Eres directo, amigable y sabes del producto.

INFORMACIÓN DEL PRODUCTO:

- Pollo cocinado al grill para darle ese delicioso sabor que usted merece
- Pollo importado de alta calidad, el mismo tipo que usan los mejores restaurante
- Cocinado al grill aquí mismo en Santo Domingo
- Cada unidad tiene aproximadamente 200g de pollo cocido con ~55g de proteína
- Sabores: Sin Sabor, Pomodoro, Pesto, Aglio, Teriyaki, Barbecue
- NO calentar en el envase plástico. Pasar el pollo a un plato antes de calentar.
- Dura 6 días en la nevera después de descongelado. Nunca romper la cadena de frío
- Se puede congelar. Para descongelar, poner en la nevera un día antes de comer.
- El pack de 5 resuelve tu proteína toda la semana.

PRECIOS:

- Pack de 3 unidades: RD\$900
- Pack de 5 unidades: RD\$1,500
- Pack de 10 unidades: RD\$3,000
- Delivery: RD\$120

ENTREGAS:

- Solo en Santo Domingo por el momento
- Pedidos antes de las 2pm se entregan en 3-4 horas
- Pedidos después de las 2pm se entregan al día siguiente
- Solo tarjeta por el momento

CANCELACIONES:

- Una vez la orden está creada y facturada no se puede cancelar ni cambiar

ALÉRGENOS:

- Ningún producto contiene nuez

INSTRUCCIONES:

- Si te preguntan cómo ordenar, diles que escriban "ordenar" o que elijan un pack
- Responde SIEMPRE en español dominicano natural
- Sé breve. Máximo 3-4 líneas por respuesta.
- Termina respuestas con energía positiva y un emoji relevante
- NUNCA inventes información sobre el producto.

`;

```
// =====  
// WEBHOOK VERIFICATION  
// =====  
app.get("/webhook", (req, res) => {  
  const mode      = req.query["hub.mode"];
```

```

const token      = req.query["hub.verify_token"];
const challenge = req.query["hub.challenge"];
if (mode === "subscribe" && token === WHATSAPP_VERIFY_TOKEN) {
  res.status(200).send(challenge);
} else {
  res.sendStatus(403);
}
});

// =====
// PRIVACY PAGE
// =====
app.get("/privacy", (req, res) => {
  res.send("<h1>Chef Papi Privacy Policy</h1><p>We collect your name, phone numbe
});

// =====
// MAIN WEBHOOK
// =====
app.post("/webhook", async (req, res) => {
  res.sendStatus(200);
  try {
    const message = req.body?.entry?.[0]?.changes?.[0]?.value?.messages?.[0];
    if (!message) return;

    const phone = message.from;
    const type = message.type;

    let input = "";
    if (type === "text") {
      input = message.text.body.trim();
    } else if (type === "interactive") {
      if (message.interactive.type === "list_reply") {
        input = message.interactive.list_reply.id;
      }
      if (message.interactive.type === "button_reply") {
        input = message.interactive.button_reply.id;
      }
    }

    await handleMessage(phone, input);
  } catch (err) {
    console.error("Webhook error:", err);
  }
});

// =====

```

```

// CORE MESSAGE HANDLER
// =====
async function handleMessage(phone, input) {
  let session = await getSession(phone);

  // — SESSION TIMEOUT CHECK —————
  if (session) {
    const expired = await isSessionExpired(session);
    if (expired) {
      await createSession(phone);
      session = await getSession(phone);
      await sendMessage(phone, "Diay, te quedaste dormido 😴 ;Empecemos de nuevo!");
      await sendZoneCheck(phone);
      return;
    }
  }

  // — NEW CUSTOMER — ask zone first —————
  if (!session) {
    await createSession(phone);
    await sendZoneCheck(phone);
    return;
  }

  // — HUMAN HANDOFF —————
  if (
    input.toLowerCase().includes("humano") ||
    input.toLowerCase().includes("hablar con") ||
    input === "human"
  ) {
    await sendHumanHandoff(phone);
    return;
  }

  // — ROUTE BY STATE —————
  switch (session.state) {

    case "AWAITING_ZONE": {
      const normalized = input.toLowerCase().trim();
      const isYes = input === "zone_yes" || ["si", "sí", "yes", "sd", "sto domingo", "
const isNo = input === "zone_no" || ["no", "nope", "fuera", "no estoy", "nel"
      if (isYes) {
        await updateSession(phone, { state: "AWAITING_PACK" });
        await sendWelcome(phone);
      } else if (isNo) {
        await sendMessage(phone,
          `Ay, qué pena 😞 Por el momento solo entregamos en *Santo Domingo*.\n\n

```

```

        `;Pronto llegamos a más ciudades! Mantente pendiente 🍌`
    );
    await createSession(phone);
} else {
    await sendZoneCheck(phone);
}
break;
}

case "AWAITING_PACK":
    if (["3","5","10"].includes(input)) {
        await handlePackSelection(phone, session, input);
    } else if (isQuestion(input)) {
        const answer = await askAI(input);
        await sendMessage(phone, answer);
        await sendWelcome(phone);
    } else {
        await sendWelcome(phone);
    }
    break;

case "AWAITING_FLAVOR":
    if (isQuestion(input) && !Object.keys(FLAVORS).includes(input)) {
        const answer = await askAI(input);
        await sendMessage(phone, answer);
        await sendMessage(phone, "Dicho eso, sigamos con tu orden 👉");
    } else {
        await handleFlavorSelection(phone, session, input);
    }
    break;

case "AWAITING_QTY":
    if (isQuestion(input) && isNaN(parseInt(input))) {
        const answer = await askAI(input);
        await sendMessage(phone, answer);
        await sendMessage(phone, "Dicho eso, ¿cuántas unidades quieres? Escribe e");
    } else {
        await handleQtyInput(phone, session, input);
    }
    break;

case "AWAITING_CONFIRM":
    await handleConfirmation(phone, session, input);
    break;

case "AWAITING_NCF":
    await handleNCFSelection(phone, session, input);

```

```

        break;

    case "AWAITING_RNC":
        await handleRNCInput(phone, session, input);
        break;

    case "AWAITING_NAME":
        await handleNameInput(phone, session, input);
        break;

    case "AWAITING_ADDRESS":
        await handleAddressInput(phone, session, input);
        break;

    case "AWAITING_PAYMENT":
        await sendMessage(phone,
            `⌚ Esperando tu pago. Usa este link:\n\n${session.pending_order?.payment}`
            `Si tuviste algún problema escríbenos y te ayudamos 🤖`
        );
        break;

    case "DONE":
        if (input === "reorder" || input === "new_order") {
            await handleReorderOrNew(phone, session, input);
        } else if (isQuestion(input)) {
            const answer = await askAI(input);
            await sendMessage(phone, answer);
            await sendHumanButton(phone);
        } else {
            await sendDoneButtons(phone);
        }
        break;

    default:
        await createSession(phone);
        await sendZoneCheck(phone);
}

}

// =====
// STATE HANDLERS
// =====

async function handlePackSelection(phone, session, input) {
    const size = parseInt(input);
    if (![3, 5, 10].includes(size)) {
        await sendWelcome(phone);
    }
}

```

```

    return;
  }
  await updateSession(phone, {
    state: "AWAITING_FLAVOR",
    pending_order: { pack_size: size, selections: [], price: PRICES[size] },
  });
  await sendMessage(phone,
    `Pack de ${size} unidades 🍋\n` +
    `Precio: RD${PRICES[size].toLocaleString()} + RD$120 delivery\n\n` +
    `¿Qué sabor quieres agregar? Te quedan *${size} unidades*.`
  );
  await sendFlavorList(phone, size);
}

async function handleFlavorSelection(phone, session, input) {
  if (!FLAVORS[input]) {
    const unidadesLeft = session.pending_order.pack_size - session.pending_order.
    await sendMessage(phone, "Selecciona un sabor de la lista 📌");
    await sendFlavorList(phone, unidadesLeft);
    return;
  }
  const flavor = FLAVORS[input];
  const unidadesLeft = session.pending_order.pack_size - session.pending_order.se
  await updateSession(phone, {
    state: "AWAITING_QTY",
    pending_order: { ...session.pending_order, current_flavor: input },
  });
  await sendMessage(phone,
    `${flavor.emoji} *${flavor.title}* - ¿cuántas unidades quieres? (te quedan *$
  );
}

async function handleQtyInput(phone, session, input) {
  const qty = parseInt(input);
  const order = session.pending_order;
  const unidadesLeft = order.pack_size - order.selections.length;
  const flavor = FLAVORS[order.current_flavor];

  if (!qty || qty < 1 || isNaN(qty)) {
    await sendMessage(phone, `Escribe un número entre 1 y ${unidadesLeft} 📌`);
    return;
  }
  if (qty > unidadesLeft) {
    await sendMessage(phone,
      `Solo te quedan *${unidadesLeft} unidades*. ¿Cuántas ${flavor.emoji} ${flav
    );
    return;
  }

```

```

}

const newSelections = [...order.selections, ...Array(qty).fill(order.current_fl
const remaining      = order.pack_size - newSelections.length;

if (remaining === 0) {
  const updatedOrder = { ...order, selections: newSelections, current_flavor: n
  await updateSession(phone, { state: "AWAITING_CONFIRM", pending_order: update
  const summary = buildSummary(newSelections);
  await sendMessage(phone,
    `✅ ¡Tu pack está listo!\n\n${summary}\n\n` +
    `Subtotal: RD${PRICES[order.pack_size].toLocaleString()}\n` +
    `Delivery: RD$120\n` +
    `*TOTAL: RD${(PRICES[order.pack_size] + 120).toLocaleString()}*`
  );
  await sendConfirmButtons(phone);
} else {
  const updatedOrder = { ...order, selections: newSelections, current_flavor: n
  await updateSession(phone, { state: "AWAITING_FLAVOR", pending_order: updated
  await sendMessage(phone, `✅ Agregado. Te quedan ${remaining} unidades*.\n¿C
  await sendFlavorList(phone, remaining);
}
}

async function handleConfirmation(phone, session, input) {
  if (input === "confirm") {
    await updateSession(phone, { state: "AWAITING_NCF" });
    await sendNCFButtons(phone);
  } else if (input === "change") {
    const size = session.pending_order.pack_size;
    await updateSession(phone, {
      state: "AWAITING_FLAVOR",
      pending_order: { pack_size: size, selections: [], price: PRICES[size] },
    });
    await sendMessage(phone, `Ta bien! Empecemos de nuevo 🔄\nTe quedan ${size}
    await sendFlavorList(phone, size);
  }
}

async function handleNCFSelection(phone, session, input) {
  if (input === "ncf_b02") {
    await updateSession(phone, {
      pending_order: { ...session.pending_order, ncf_type: "B02" }
    });
    await continueToCustomerInfo(phone, session);
  } else if (input === "ncf_b01") {
    await updateSession(phone, {

```



```

        state: "AWAITING_RNC",
        pending_order: { ...session.pending_order, ncf_type: "B01" }
    });
    await sendMessage(phone, "📞 Perfecto. ¿Cuál es el *RNC* de tu empresa?");
} else {
    await sendNCFButtons(phone);
}
}

async function handleRNCInput(phone, session, input) {
    if (input.length < 9) {
        await sendMessage(phone, "El RNC debe tener al menos 9 dígitos. Escríbelo de return;
    }
    await updateSession(phone, {
        pending_order: { ...session.pending_order, rnc: input }
    });
    await continueToCustomerInfo(phone, session);
}

async function continueToCustomerInfo(phone, session) {
    const customer = await getCustomer(phone);
    if (!customer) {
        await updateSession(phone, { state: "AWAITING_NAME" });
        await sendMessage(phone, "¿Qué nivel! Antes de continuar, ¿cuál es tu nombre
    } else {
        await updateSession(phone, { state: "AWAITING_ADDRESS" });
        await sendAddressConfirmButtons(phone, customer.delivery_address);
    }
}

async function handleNameInput(phone, session, input) {
    if (input.length < 2) {
        await sendMessage(phone, "Escribe tu nombre completo por favor 📌");
        return;
    }
    await updateSession(phone, {
        state: "AWAITING_ADDRESS",
        pending_order: { ...session.pending_order, customer_name: input },
    });
    await sendMessage(phone,
        `Bienvenido ${input}! 🙌\n¿Cuál es tu dirección de entrega?\n\n_(Calle, número
    );
}

async function handleAddressInput(phone, session, input) {
    if (input === "same_address") {

```

```

    const customer = await getCustomer(phone);
    await createOrderAndCharge(phone, session, customer.delivery_address);
    return;
}
if (input === "new_address") {
    await updateSession(phone, { state: "AWAITING_ADDRESS" });
    await sendMessage(phone, "¿Cuál es la nueva dirección de entrega?\n\n(Calle,
    return;
}
if (input.length < 5) {
    await sendMessage(phone, "Escribe tu dirección completa por favor (calle, sec
    return;
}
const name = session.pending_order?.customer_name;
if (name) {
    await upsertCustomer(phone, { name, delivery_address: input });
}
await createOrderAndCharge(phone, session, input);
}

async function handleReorderOrNew(phone, session, input) {
    if (input === "reorder") {
        const lastOrder = await getLastOrder(phone);
        if (!lastOrder) {
            await updateSession(phone, { state: "AWAITING_PACK", pending_order: null });
            await sendWelcome(phone);
            return;
        }
        const summary = buildSummary(lastOrder.flavors);
        const customer = await getCustomer(phone);
        await updateSession(phone, {
            state: "AWAITING_CONFIRM",
            pending_order: {
                pack_size: lastOrder.pack_size,
                selections: lastOrder.flavors,
                price: PRICES[lastOrder.pack_size],
                customer_name: customer.name,
                delivery_address: customer.delivery_address,
            },
        });
        await sendMessage(phone,
            `Tu última orden:\n\n${summary}\n\n` +
            `Subtotal: RD${PRICES[lastOrder.pack_size].toLocaleString()}\n` +
            `Delivery: RD$120\n` +
            `*TOTAL: RD${(PRICES[lastOrder.pack_size] + 120).toLocaleString()}*`
        );
        await sendConfirmButtons(phone);
    }
}

```

```

    } else {
      await updateSession(phone, { state: "AWAITING_PACK", pending_order: null });
      await sendWelcome(phone);
    }
  }

// =====
// ORDER CREATION & PAYMENT
// =====
async function createOrderAndCharge(phone, session, address) {
  const order = session.pending_order;
  const customer = await getCustomer(phone);

  const { data: newOrder } = await supabase
    .from("orders")
    .insert({
      customer_id: customer.id,
      pack_size: order.pack_size,
      total_price: order.price + 120,
      status: "pending",
      delivery_address: address,
    })
    .select()
    .single();

  const items = order.selections.map((flavor, i) => ({
    order_id: newOrder.id,
    unit_number: i + 1,
    flavor,
  }));
  await supabase.from("order_items").insert(items);

  const paymentLink = `https://pay.cardnet.com.do/checkout?order=${newOrder.id}&a

  await supabase.from("orders").update({ cardnet_ref: `CN-${newOrder.id}` }).eq("

  await updateSession(phone, {
    state: "AWAITING_PAYMENT",
    pending_order: { ...order, order_id: newOrder.id, payment_link: paymentLink }
  });

  const orderNum = `CP-${String(newOrder.id).padStart(5, "0")}`;

  await sendMessage(phone,
    🎉 ;Brutal! Tu orden está lista.\n\n` +
    📦 Orden: *${orderNum}*\n\n` +
    📍 Entrega: ${address}\n\n` +

```

```

        `🇵🇪 *Link de pago:*\n${paymentLink}\n\n` +
        `🕒 Tienes 30 minutos para pagar.\n\n` +
        `⚠️ _Al calentar el pollo, pásalo a un plato primero. No calentar en el envase`
        `;Gracias por elegir Chef Papi! 🍳🍗`
    );
}

// =====
// CARDNET PAYMENT WEBHOOK
// =====
app.post("/payment-confirm", async (req, res) => {
    res.sendStatus(200);
    try {
        const { order_id, status, reference } = req.body;
        if (status !== "APPROVED") return;

        await supabase.from("orders").update({ status: "paid", cardnet_ref: reference

        const { data: order } = await supabase
            .from("orders")
            .select("*, customers(*), order_items(*)")
            .eq("id", order_id)
            .single();

        const phone = order.customers.whatsapp_phone;
        const orderNum = `CP-${String(order_id).padStart(5, "0")}`;
        const summary = buildSummary(order.order_items.map(i => i.flavor));

        await sendMessage(phone,
            `✅ *_Pago recibido!* Gracias ${order.customers.name} 🙌\n\n` +
            `📄 *${orderNum}*\n${summary}\n\n` +
            `🛵 Tu pedido está siendo preparado.\n\n` +
            `Te avisamos cuando esté en camino.\n\n` +
            `⚠️ _Al calentar, pasa el pollo a un plato primero._`
        );

        await updateSession(phone, { state: "DONE", pending_order: null });
        await sendDoneButtons(phone);

    } catch (err) {
        console.error("Payment confirm error:", err);
    }
});

// =====
// AI QUESTION HANDLER
// =====

```

```

async function askAI(question) {
  try {
    const response = await fetch("https://api.anthropic.com/v1/messages", {
      method: "POST",
      headers: {
        "Content-Type": "application/json",
        "x-api-key": ANTHROPIC_API_KEY,
        "anthropic-version": "2023-06-01",
      },
      body: JSON.stringify({
        model: "claude-haiku-4-5-20251001",
        max_tokens: 300,
        system: CHEF_PAPI_KNOWLEDGE,
        messages: [{ role: "user", content: question }],
      }),
    });
    const data = await response.json();
    return data.content?.[0]?.text || "Díay, no tengo esa info ahora mismo 🤔 Esc";
  } catch (err) {
    console.error("AI error:", err);
    return "Díay, algo pasó de mi lado 🤔 Escríbele al equipo directamente 📌";
  }
}

function isQuestion(input) {
  if (!input) return false;
  const commands = [
    "3", "5", "10", "confirm", "change", "reorder", "new_order",
    "same_address", "new_address", "ncf_b01", "ncf_b02", "human",
    "zone_yes", "zone_no"
  ];
  if (commands.includes(input)) return false;
  if (Object.keys(FLAVORS).includes(input)) return false;
  if (!isNaN(parseInt(input))) return false;
  return true;
}

// =====
// SESSION TIMEOUT
// =====

async function isSessionExpired(session) {
  if (!session?.updated_at) return false;
  const lastActive = new Date(session.updated_at);
  const now = new Date();
  const minutes = (now - lastActive) / 1000 / 60;
  return minutes >= SESSION_TIMEOUT_MINUTES;
}

```

```
// =====
// WHATSAPP SENDERS
// =====

async function sendWA(phone, body) {
  await fetch(`https://graph.facebook.com/v19.0/${WHATSAPP_PHONE_ID}/messages`, {
    method: "POST",
    headers: {
      "Authorization": `Bearer ${WHATSAPP_TOKEN}`,
      "Content-Type": "application/json",
    },
    body: JSON.stringify({ messaging_product: "whatsapp", to: phone, ...body }),
  });
}

async function sendMessage(phone, text) {
  await sendWA(phone, { type: "text", text: { body: text } });
}

async function sendZoneCheck(phone) {
  await sendWA(phone, {
    type: "interactive",
    interactive: {
      type: "button",
      header: { type: "text", text: "Chef Papi 🍗" },
      body: { text: "¡Hola! Bienvenido a Chef Papi 🧑🍳\n\nAntes de empezar - ¿es",
        footer: { text: "Solo entregamos en Santo Domingo por ahora." },
        action: {
          buttons: [
            { type: "reply", reply: { id: "zone_yes", title: "✅ Sí, estoy en SD" } },
            { type: "reply", reply: { id: "zone_no", title: "❌ No, estoy fuera" } },
          ],
        },
      },
    },
  });
}

async function sendWelcome(phone) {
  const customer = await getCustomer(phone);
  const greeting = customer
    ? `¡Qué lo qué ${customer.name}! 🙌`
    : "¡Brutal! Vamos a hacer tu orden 🍗";

  await sendWA(phone, {
    type: "interactive",
    interactive: {
      type: "button",

```

```

header: { type: "text", text: "Chef Papi 🍗" },
body:   { text: `${greeting}\nPollo al grill, listo para tu semana.\n\n¿Qué
footer: { text: "Entrega fría. Come toda la semana." },
action: {
  buttons: [
    { type: "reply", reply: { id: "3", title: "Pack 3 - RD$900" } },
    { type: "reply", reply: { id: "5", title: "Pack 5 - RD$1,500" } },
    { type: "reply", reply: { id: "10", title: "Pack 10 - RD$3,000" } },
  ],
},
});
}

```

```

async function sendFlavorList(phone, unidadesLeft) {
  await sendWA(phone, {
    type: "interactive",
    interactive: {
      type: "list",
      header: { type: "text", text: `Unidades disponibles: ${unidadesLeft}` },
      body:   { text: "¿Qué sabor quieres agregar?" },
      action: {
        button: "Ver sabores 🍗",
        sections: [{
          title: "Sabores disponibles",
          rows: Object.entries(FLAVORS).map(([id, f]) => ({
            id,
            title: `${f.emoji} ${f.title}`,
          })),
        ]},
      },
    },
  });
}

```

```

async function sendConfirmButtons(phone) {
  await sendWA(phone, {
    type: "interactive",
    interactive: {
      type: "button",
      body: { text: "¿Confirmamos esta orden?" },
      action: {
        buttons: [
          { type: "reply", reply: { id: "confirm", title: "✅ Confirmar" } },
          { type: "reply", reply: { id: "change", title: "🔄 Cambiar" } },
        ],
      },
    },
  });
}

```

```

    },
  });
}

async function sendNCFButtons(phone) {
  await sendWA(phone, {
    type: "interactive",
    interactive: {
      type: "button",
      body: { text: "🧾 ¿Necesitas factura con NCF?" },
      action: {
        buttons: [
          { type: "reply", reply: { id: "ncf_b02", title: "Consumidor final" } },
          { type: "reply", reply: { id: "ncf_b01", title: "Con RNC empresarial" } },
        ],
      },
    },
  });
}

```

```

async function sendAddressConfirmButtons(phone, address) {
  await sendWA(phone, {
    type: "interactive",
    interactive: {
      type: "button",
      body: { text: `¿Entregamos en la misma dirección?\n\n📍 ${address}` },
      action: {
        buttons: [
          { type: "reply", reply: { id: "same_address", title: "✅ Sí, misma dire" },
          { type: "reply", reply: { id: "new_address", title: "📍 Nueva direcció" },
        ],
      },
    },
  });
}

```

```

async function sendDoneButtons(phone) {
  await sendWA(phone, {
    type: "interactive",
    interactive: {
      type: "button",
      body: { text: "¿Qué más puedo hacer por ti? 🙏" },
      action: {
        buttons: [
          { type: "reply", reply: { id: "reorder", title: "🔄 Repetir última or" },
          { type: "reply", reply: { id: "new_order", title: "🛒 Nueva orden" },
          { type: "reply", reply: { id: "human", title: "💬 Hablar con alguie" },
        ],
      },
    },
  });
}

```



```

        ],
      },
    },
  });
}

async function sendHumanButton(phone) {
  await sendWA(phone, {
    type: "interactive",
    interactive: {
      type: "button",
      body: { text: "¿Necesitas hablar con alguien del equipo?" },
      action: {
        buttons: [
          { type: "reply", reply: { id: "human", title: "💬 Contactar equipo" } }
        ],
      },
    },
  });
}

async function sendHumanHandoff(phone) {
  await sendMessage(phone,
    `¡Claro! Aquí te conecto con el equipo de Chef Papi 🍳\n\n` +
    `📱 Escribenos directo:\nwa.me/18098831687\n\n` +
    `Estamos disponibles para ayudarte. ¡Hasta luego! 🍗`
  );
}

// =====
// DATABASE HELPERS
// =====

async function getSession(phone) {
  const { data, error } = await supabase.from("sessions").select("*").eq("phone",
    if (error) console.error("getSession error:", error);
  return data || null;
}

async function createSession(phone) {
  const { data } = await supabase
    .from("sessions")
    .upsert({ phone, state: "AWAITING_ZONE", pending_order: null, updated_at: new
    .select()
    .single();
  return data;
}

```

```

async function updateSession(phone, updates) {
  await supabase
    .from("sessions")
    .update({ ...updates, updated_at: new Date().toISOString() })
    .eq("phone", phone);
}

async function getCustomer(phone) {
  const { data, error } = await supabase.from("customers").select("*").eq("whatsa
  if (error) console.error("getCustomer error:", error);
  return data || null;
}

async function upsertCustomer(phone, { name, delivery_address }) {
  await supabase.from("customers").upsert({ whatsapp_phone: phone, name, delivery
}

async function getLastOrder(phone) {
  const customer = await getCustomer(phone);
  if (!customer) return null;
  const { data } = await supabase
    .from("orders")
    .select("*, order_items(*)")
    .eq("customer_id", customer.id)
    .eq("status", "paid")
    .order("created_at", { ascending: false })
    .limit(1)
    .single();
  if (!data) return null;
  return { pack_size: data.pack_size, flavors: data.order_items.map(i => i.flavor
}

// =====
// UTILS
// =====
function buildSummary(selections) {
  const counts = {};
  for (const f of selections) counts[f] = (counts[f] || 0) + 1;
  return Object.entries(counts)
    .map(([id, qty]) => `${FLAVORS[id].emoji} ${FLAVORS[id].title} x${qty}`)
    .join("\n");
}

// =====
// START
// =====
app.listen(PORT, () => console.log(`🍌 Chef Papi v9 running on port ${PORT}`));

```

